CYLAB SECURITY ACADEMY

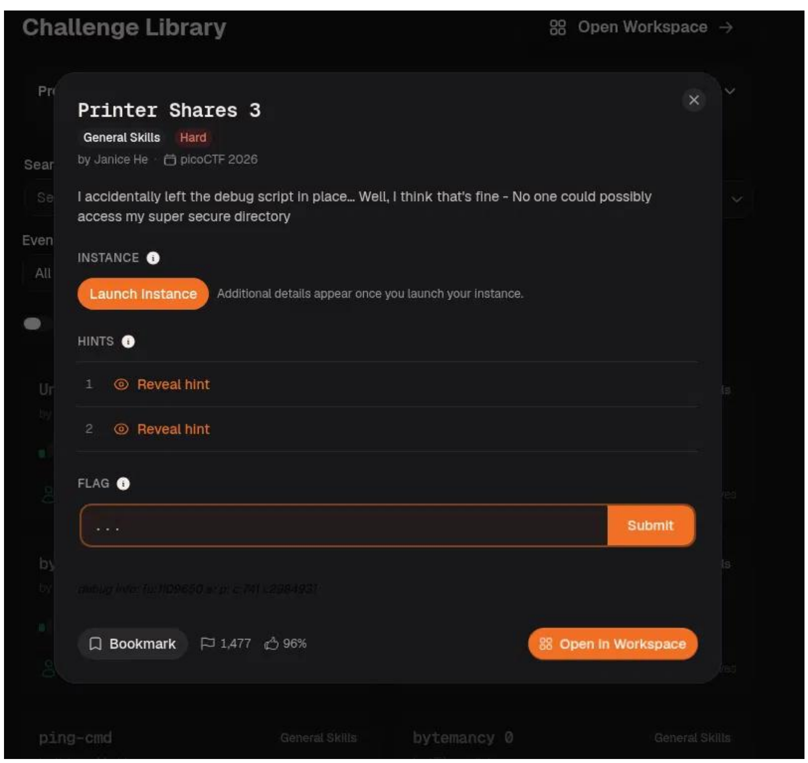
Challenge Write-Up: Printer Shares 3

Challenge Information

Student Name	Keyzia Joy Pascua
Section / Class	BSIT 3 - Networking
Challenge Name	Printer Shares 3
Category	General Skills
Points / Difficulty	Hard
Date Solved	August 15, 2026
Flag Format Given	picoCTF{...}

1. Challenge Description

The challenge exposes an SMB server (dolphin-cove.picoctf.net, on a non-default port) hosting two shares. One, "shares," is open to anonymous/guest logins. The other, "secure-shares," is labeled as internal printer use only, and the prompt jokes that a debug script was accidentally left in place but that the "secure" directory couldn't possibly be reached anyway. The goal is to read a flag that lives inside that restricted secure-shares location using only what's reachable through the public, guest-accessible share.



2. Initial Analysis / Reconnaissance

I started with a quick nc -vz scan of the given host and port to confirm the service was up before touching anything else. I then used smbclient -L with -N (guest, no password) to anonymously list the shares, which showed a "Public Share With Guests" named shares and a printer-labeled share named secure-shares explicitly called out as internal-only. Trying to force an SMB1 workgroup listing on the same connection was refused, telling me the server wasn't going to hand over anything extra just by asking — whatever the intended path was, it had to run through content placed on the public share. Given the two-tier share setup and the "debug script left in place" flavor text, I categorized this as a General Skills / SMB misconfiguration challenge, where the real vulnerability was a script on the public share that gets executed by something with more access than a guest SMB session.

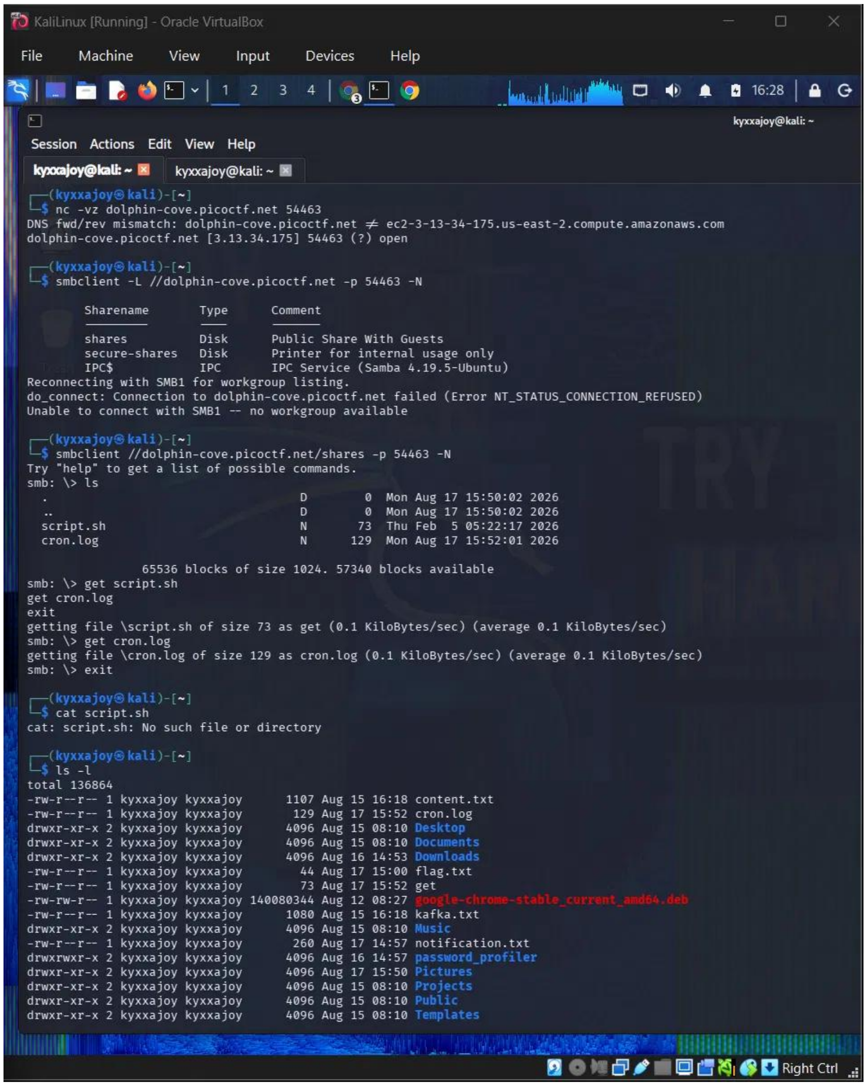
3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
nc -vz dolphin-cove.picoctf.net 54463	Confirm the given host/port was reachable before doing anything else.	Port 54463 reported open, confirming a listening service.
smbclient -L //dolphin-cove.picoctf.net -p 54463 -N	Anonymously (guest) enumerate the SMB shares on the box.	Two shares found: "shares" (Public Share With Guests) and "secure-shares" (Printer, internal usage only), plus IPC$.
smbclient //dolphin-cove.picoctf.net/shares -p 54463 -N	Connect as guest to the public share to see what was left there.	Directory listing showed script.sh (73 bytes) and cron.log (129 bytes).
get / put (inside smbclient)	Download the debug script/log, then upload a modified script back to the share.	Pulled script.sh and cron.log locally; later pushed an edited script.sh so the server's cron job would run it.
cat / file / nano	Read the downloaded files and edit the script to add recon / exfiltration commands.	file confirmed a Bash script; nano added a find command, then a cat command targeting the flag.

4. Solution Walkthrough

Step 1

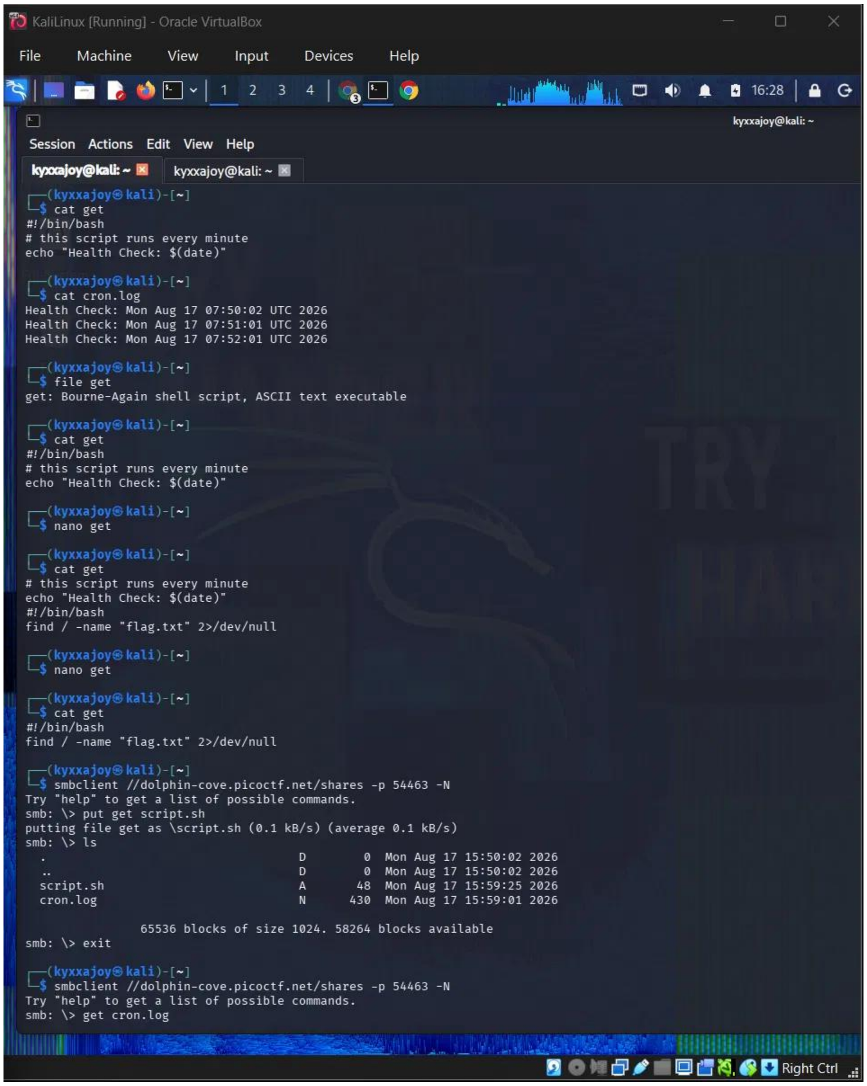
Enumerate the SMB shares and pull down what's on the public one. After confirming the port was open and listing the shares anonymously, I connected directly to the public shares share with smbclient (still -N for guest access). Inside I found only script.sh (73 bytes) and cron.log (129 bytes) — clearly leftovers from an automated process on the box. I downloaded both to inspect locally.



Port scan, anonymous SMB share listing (shares / secure-shares), and downloading script.sh and cron.log.

Step 2

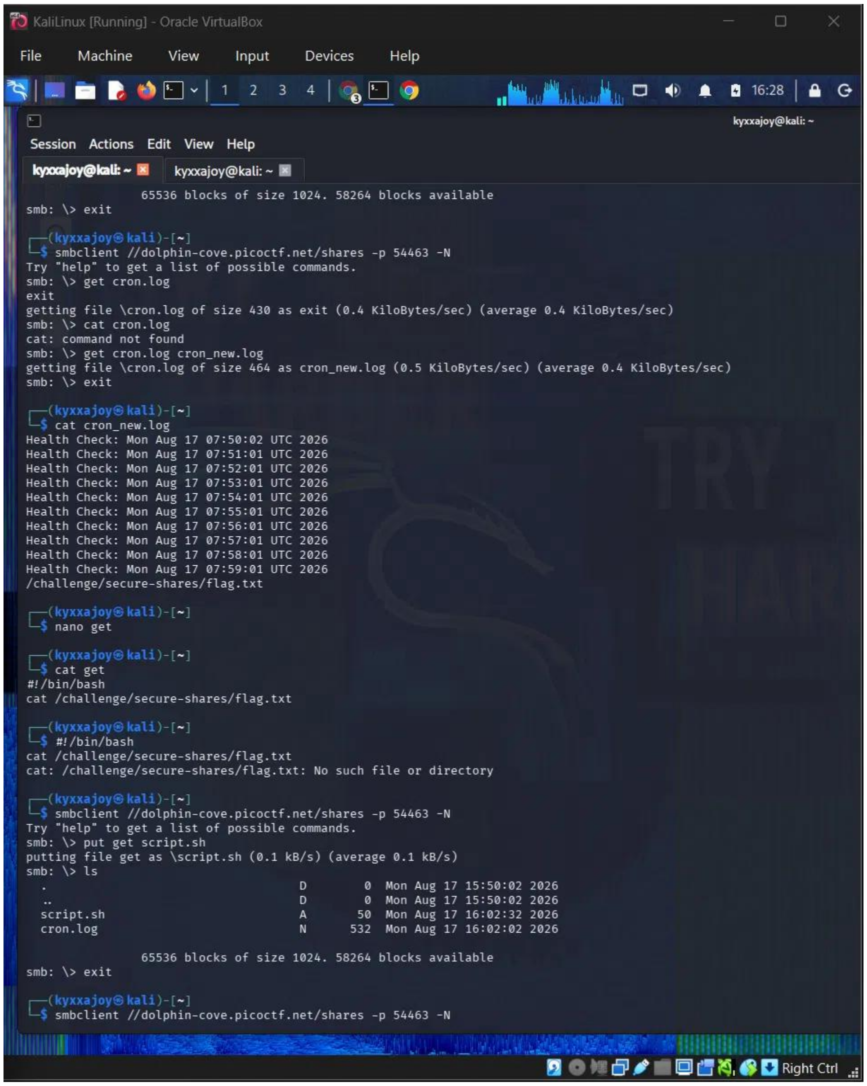
Understand the script and weaponize it. The script simply echoed "Health Check: $(date)" into cron.log, and cron.log held one such line per minute — confirming this script runs automatically via a cron job on the server, not over my guest SMB session. That's the privilege boundary I needed: whatever account runs that cron job likely has filesystem access I don't have as an anonymous guest. Since the share was writable, I overwrote script.sh with my own commands and let the server's cron job run them for me. First move: I edited the script with nano to add find / -name "flag.txt" 2>/dev/null, hoping to locate the flag anywhere on the server, including inside the restricted secure-shares path, then re-uploaded it with put.



Inspecting script.sh / cron.log, editing the script in nano to add a find command, and pushing it back with put.

Step 3

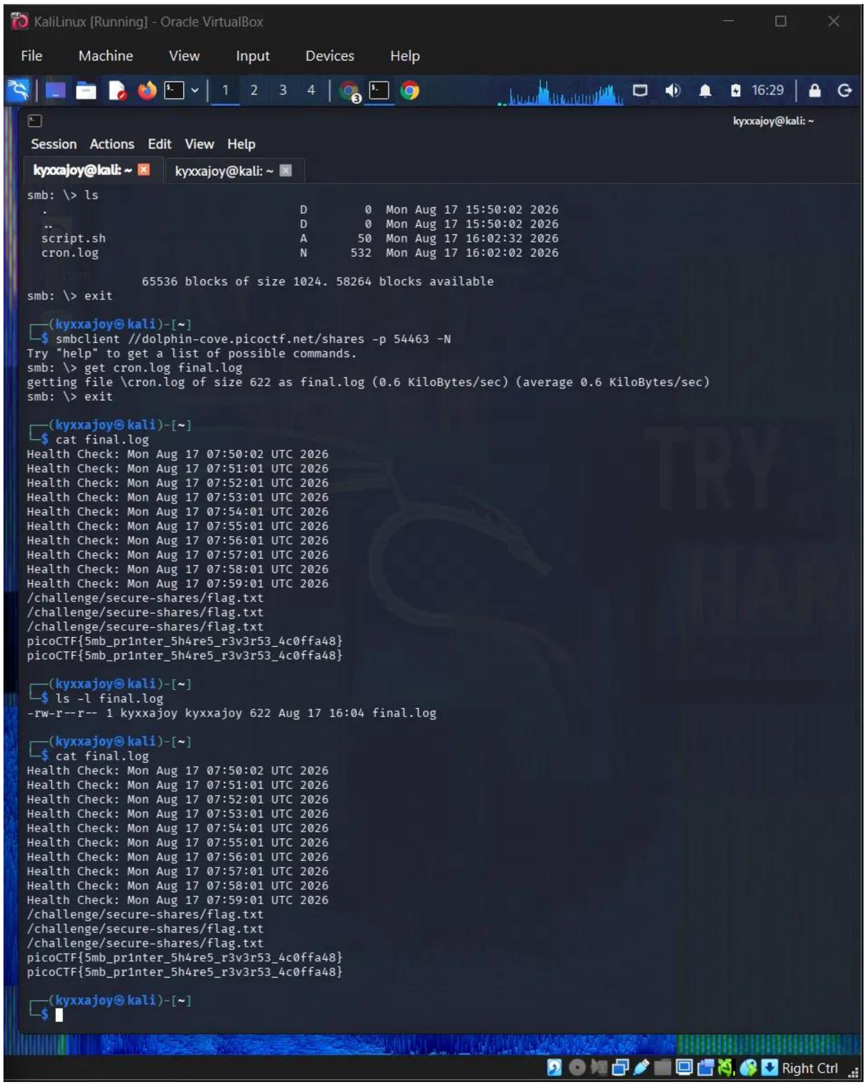
Wait for cron to run the modified script, then read the results. After waiting for the next minute, I pulled a fresh copy of cron.log. The new entries showed the ordinary health-check lines followed by a new line from my injected find command: /challenge/secure-shares/flag.txt. This confirmed the cron job runs with enough privilege to see inside secure-shares, and gave me the exact path to the flag file — a path I couldn't browse to directly over my guest SMB session.



Updated cron.log after the cron job re-ran, revealing /challenge/secure-shares/flag.txt from the find output.

Additional steps (add as needed)

Read the flag file directly through the cron job. With the exact path in hand, I edited the script again to replace the find command with cat /challenge/secure-shares/flag.txt and re-uploaded it the same way. (I briefly tried that same cat command on my own Kali box out of habit — it failed there, as expected, since the path only exists on the challenge server.) After the next cron tick, the downloaded cron.log contained the contents of the secure flag file, printed by two consecutive cron runs.



Final cron.log showing the cat command output — the flag captured through the cron job's access to secure-shares.

5. Flag

```
picoCTF{5mb_pr1nter_5h4re5_r3v3r53_4c0ffa48}
```

6. Key Concept / What I Learned

A world-writable file isn't risky only because of what an attacker can put in it — it's risky because of what executes it. The SMB guest account never had access to secure-shares, but a cron job on the server did, and that cron job blindly re-ran whatever was sitting in the public share's script.sh once a minute. By overwriting a script that a higher-privileged, automated process trusted, I effectively borrowed that process's access instead of breaking the SMB permission boundary directly — a confused-deputy / privilege-escalation-via-scheduled-task pattern, and a reminder that "no one could access it directly" isn't the same as "nothing with access to it can be manipulated."

7. References

smbclient(1) man page — connecting, listing shares, and using get/put over SMB with guest (-N) authentication.

picoCTF 2026 Challenge Library — "Printer Shares 3" challenge page (General Skills, Hard), by Janice He.